

Rapport de Projet

Construction d'une Chaîne CI/CD

Intégration de SonarQube , Nexus Repository et jenkins

Projet DevOps : analyse qualité, packaging, publication et gestion des artefacts

Réaliser par : Marzouki jihed / firass ghazel

Table des matières

1	Introduction	1
2	Architecture générale de la chaîne CI/CD	1
3	Préparation du projet	1
3.1	Clonage du dépôt Git.....	1
3.2	Installation et vérification de Maven.....	2
4	Mise en place de SonarQube	2
4.1	Rôle de SonarQube.....	2
4.2	Lancement de SonarQube avec Docker.....	2
4.3	Configuration Maven pour l'analyse SonarQube.....	4
4.4	Couverture de tests avec JaCoCo.....	4
5	Analyse du frontend Angular avec SonarQube	4
5.1	Création du projet local.....	4
5.2	Résultats d'analyse.....	5
6	Analyse du backend Spring Boot avec SonarQube	6
6.1	Création et configuration du projet backend.....	6
6.2	Génération du token d'accès.....	7
6.3	Choix de la méthode d'analyse.....	8
6.4	Amélioration après ajout des tests.....	9
7	Mise en place de Nexus Repository	9
7.1	Rôle de Nexus.....	9
7.2	Packaging du backend Spring Boot.....	10
7.3	Packaging du frontend Angular.....	10
7.4	Authentification et accès à Nexus.....	12
8	Configuration des dépôts Nexus	13
8.1	Repository backend.....	13
8.2	Repository proxy.....	14

8.3	Repository group	15
8.4	Repository frontend	16
9	Problème rencontré et solution appliquée	16
10	Bilan du projet	16
11	Conclusion	17
	Références utiles	17

1 Introduction

Ce rapport présente la construction d'une chaîne CI/CD sécurisée pour un projet composé d'un backend Spring Boot et d'un frontend Angular. L'objectif principal est de mettre en place un processus fiable permettant de récupérer le code source, de compiler les applications, d'analyser leur qualité avec SonarQube, puis de publier les livrables dans Nexus Repository.

La démarche suivie permet de renforcer la qualité logicielle avant livraison. SonarQube est utilisé pour contrôler la maintenabilité, les bugs, les vulnérabilités, les duplications et la couverture des tests. Nexus est utilisé comme dépôt centralisé pour stocker les artefacts générés, notamment les fichiers `.jar` du backend et les archives `.zip` du frontend.

Objectifs du projet

- Cloner et préparer le projet applicatif depuis Git.
- Installer et vérifier l'environnement Maven nécessaire au backend Spring Boot.
- Déployer SonarQube avec Docker et configurer les projets d'analyse.
- Exécuter l'analyse qualité pour le frontend Angular et le backend Spring Boot.
- Générer les livrables applicatifs et les publier dans Nexus Repository.
- Organiser les dépôts Nexus selon les besoins : hosted, proxy et group.

2 Architecture générale de la chaîne CI/CD

La chaîne réalisée repose sur une organisation classique DevOps. Le code source est récupéré depuis un dépôt Git, les dépendances sont résolues avec Maven ou npm selon la partie du projet, les analyses sont envoyées vers SonarQube et les artefacts finaux sont publiés dans Nexus.

Composant	Rôle dans la chaîne CI/CD
Git	Centralise le code source et permet de cloner le projet localement.
Maven	Compile le backend Spring Boot, lance les tests et génère le fichier <code>.jar</code> .
npm/Angular	Installe les dépendances frontend et produit les fichiers de distribution.
SonarQube	Analyse la qualité du code, la maintenabilité, les bugs et la sécurité.
Docker	Fournit un environnement simple et reproductible pour exécuter SonarQube et nexus.
Nexus	Stocke et organise les artefacts backend et frontend.

3 Préparation du projet

3.1 Clonage du dépôt Git

La première étape consiste à récupérer le code source depuis Git. Cette opération permet de travailler sur une copie locale du projet et de préparer les phases de build, de test et d'analyse.

```
useradm@devops:~/Bureau/project-sonar-nexus$ sudo apt install maven -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
maven est déjà la version la plus récente (3.8.7-2).
0 mis à jour, 0 nouvellement installés, 0 à enlever et 383 non mis à jour.
useradm@devops:~/Bureau/project-sonar-nexus$
```

FIGURE 1 – Clonage du projet depuis le dépôt Git.

```
useradm@devops:~$ cd Bureau/
useradm@devops:~/Bureau$ git clone https://github.com/MarzoukiJihed/project-sonar-nexus.git
Clonage dans 'project-sonar-nexus'...
remote: Enumerating objects: 29, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (19/19), done.
remote: Total 29 (delta 0), reused 29 (delta 0), pack-reused 0 (from 0)
Réception d'objets: 100% (29/29), 24.92 Kio | 79.00 Kio/s, fait.
useradm@devops:~/Bureau$ cd project-sonar-nexus/
```

FIGURE 2 – Vérification de la récupération du projet en local.

3.2 Installation et vérification de Maven

Maven est indispensable pour la partie backend Spring Boot. Il permet de gérer les dépendances, de compiler le projet, d'exécuter les tests unitaires et de générer le livrable final. La vérification de l'installation confirme que l'environnement est prêt pour l'analyse et le packaging.

4 Mise en place de SonarQube

4.1 Rôle de SonarQube

SonarQube est une plateforme d'inspection continue de la qualité du code. Elle permet de détecter les erreurs, les vulnérabilités, les mauvaises pratiques, les duplications et les problèmes de maintenabilité. Dans cette chaîne CI/CD, SonarQube intervient avant la publication des livrables afin de garantir que le code respecte un niveau de qualité acceptable.

Le serveur SonarQube démarre principalement trois processus :

- **Web Server** : fournit l'interface web utilisée par les administrateurs et les développeurs.
- **Search Server** : repose sur Elasticsearch et permet d'effectuer les recherches dans l'interface.
- **Compute Engine Server** : traite les rapports d'analyse envoyés par les scanners et enregistre les résultats dans la base de données SonarQube.

La base de données SonarQube conserve la configuration de l'instance, les paramètres des projets, l'historique des analyses et les rapports de qualité.

4.2 Lancement de SonarQube avec Docker

SonarQube a été lancé via Docker sur le port 9000. Cette approche simplifie le déploiement, car elle évite une installation manuelle lourde et permet d'obtenir rapidement une instance exploitable.

```
useradm@devops: /bureau/project-sonar-nexus$ docker run -d --name sonarqube -p 9000:9000 -e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true -v sonarqube-data:/opt/sonarqube/data -v sonarqube-logs:/opt/sonarqube/logs sonarqube:latest
```

FIGURE 3 – Commande de lancement de SonarQube avec Docker sur le port 9000.

Après le démarrage, l'interface est accessible depuis le navigateur à l'adresse <http://localhost:9000>. Les identifiants initiaux utilisés sont admin/admin, puis un nouveau mot de passe est configuré lors de la première connexion.

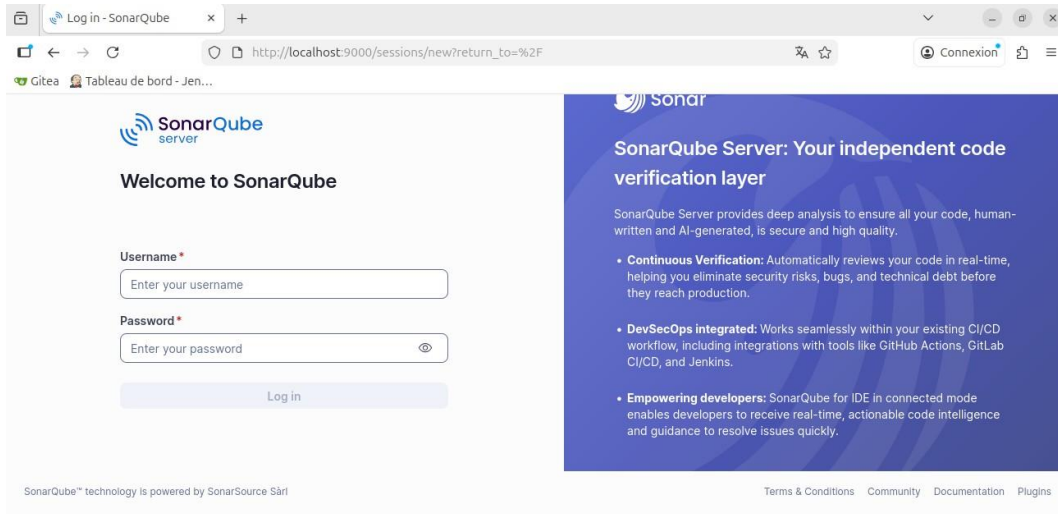


FIGURE 4 – Page de connexion SonarQube accessible sur le port 9000.

Update your password

⚠ This account should not use the default password.

Enter a new password

Old Password *

.....

Password *

.....

Confirm Password *

.....

Update

FIGURE 5 – Changement du mot de passe administrateur après la première connexion.

4.3 Configuration Maven pour l'analyse SonarQube

Pour permettre l'analyse du backend Spring Boot, le plugin SonarQube est ajouté dans le fichier pom.xml, au niveau de la section build/plugins. Cette configuration permet à Maven d'envoyer les résultats d'analyse vers le serveur SonarQube.

4.4 Couverture de tests avec JaCoCo

JaCoCo est ajouté à la partie backend afin de mesurer la couverture des tests unitaires. Il génère un rapport de couverture qui peut être lu par SonarQube pendant l'analyse Maven. Cette intégration permet de visualiser dans SonarQube le pourcentage de code testé, les lignes non couvertes et les zones du backend qui nécessitent davantage de tests.

Dans une chaîne CI/CD, JaCoCo complète donc l'analyse SonarQube : SonarQube évalue la qualité globale du code, tandis que JaCoCo fournit les métriques de couverture nécessaires pour valider la fiabilité des tests avant la livraison.

5 Analyse du frontend Angular avec SonarQube

5.1 Création du projet local

Dans SonarQube, un projet local est créé depuis le menu **Projects**, puis **Create a local project**. Cette étape déclare le projet dans la plateforme afin de recevoir les résultats d'analyse.

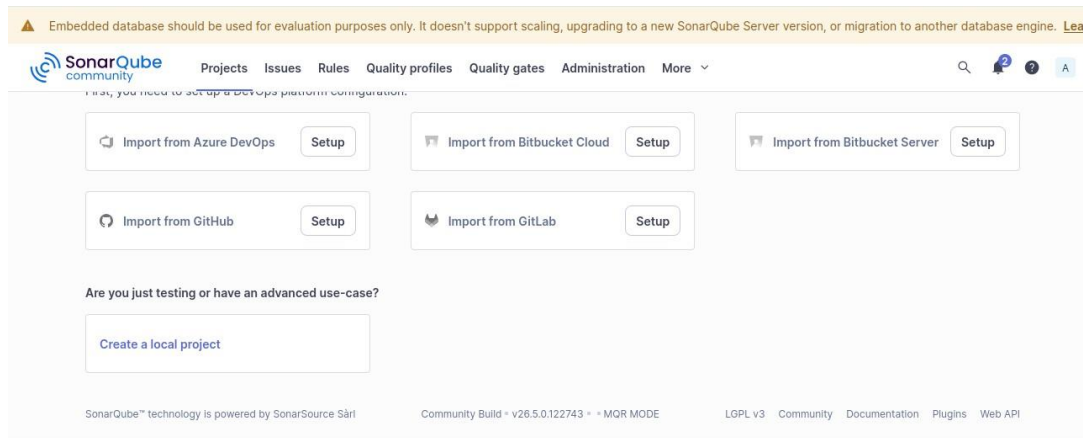


FIGURE 6 – Création d'un projet local pour le frontend Angular dans SonarQube.

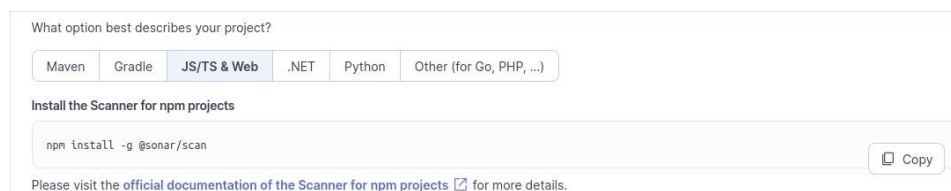


FIGURE 7 – Commande fournie par SonarQube pour analyser le frontend Angular.

```

useradm@devops:~/Bureau/ci-cd/angular-factorial-app$ sudo npm install -g @sonar/scan
[sudo] Mot de passe de useradm :

added 44 packages in 11s

6 packages are looking for funding
  run `npm fund` for details
useradm@devops:~/Bureau/ci-cd/angular-factorial-app$

```

FIGURE 8 – Exécution de la commande d'analyse SonarQube pour le frontend Angular.

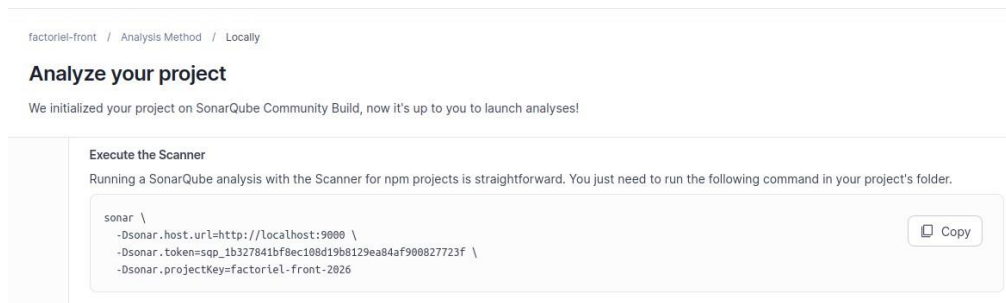


FIGURE 9 – Finalisation de l'analyse du projet frontend dans SonarQube.

5.2 Résultats d'analyse

Après exécution de l'analyse, SonarQube affiche un tableau de bord permettant de suivre l'état global du projet. Les indicateurs visibles concernent notamment la qualité globale, la maintenabilité, la fiabilité, la sécurité et les duplications.

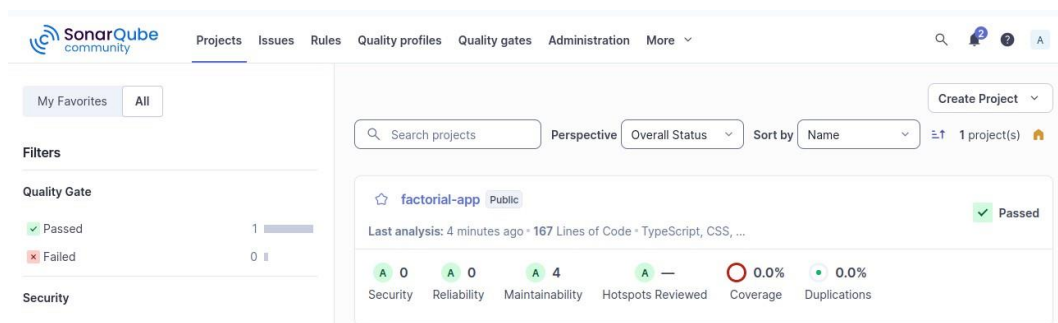


FIGURE 10 – Résultat global de l'analyse SonarQube du frontend Angular.



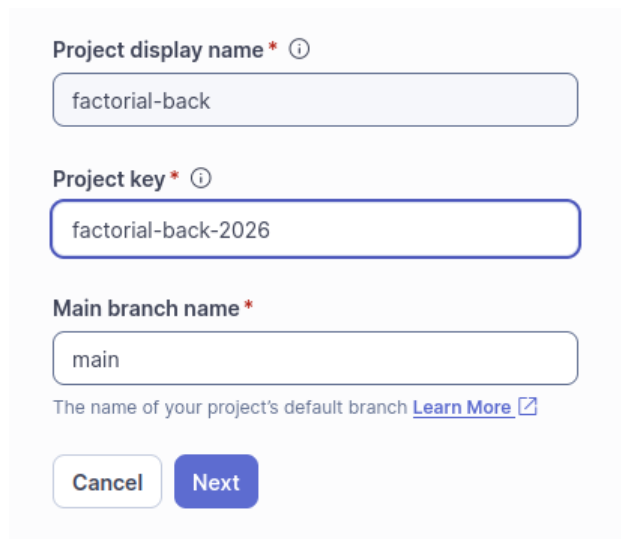
FIGURE 11 – Indicateurs de maintenabilité du frontend.

Ces résultats permettent d'identifier les points à corriger avant intégration ou livraison. La maintenabilité est particulièrement importante, car elle mesure l'effort nécessaire pour comprendre, modifier et faire évoluer le code.

6 Analyse du backend Spring Boot avec SonarQube

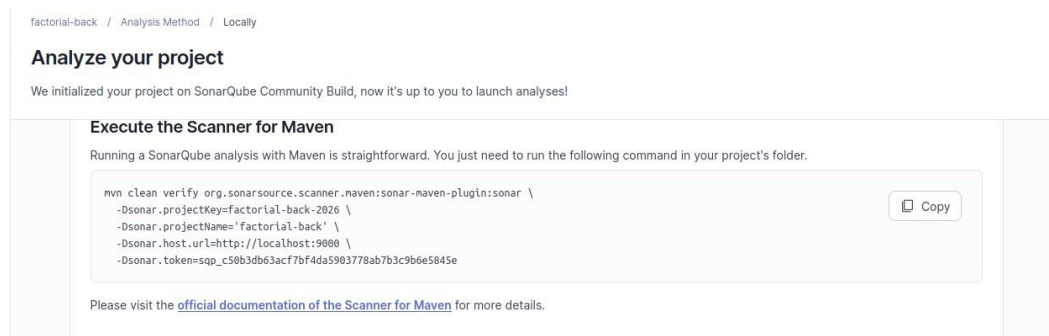
6.1 Création et configuration du projet backend

Un second projet local est créé pour le backend Spring Boot. La séparation frontend/backend permet d'obtenir des métriques indépendantes pour chaque partie de l'application.



The screenshot shows the 'Create Project' form in SonarQube. It contains three input fields: 'Project display name' with the value 'factorial-back', 'Project key' with the value 'factorial-back-2026', and 'Main branch name' with the value 'main'. Below the inputs is a link 'Learn More' and two buttons: 'Cancel' and 'Next'.

FIGURE 12 – Création du projet backend dans SonarQube.



The screenshot shows the 'Analyze your project' page in SonarQube. It includes a breadcrumb 'factorial-back / Analysis Method / Locally' and a heading 'Analyze your project'. Below the heading is a message: 'We initialized your project on SonarQube Community Build, now it's up to you to launch analyses!'. The main section is titled 'Execute the Scanner for Maven' and contains the following text: 'Running a SonarQube analysis with Maven is straightforward. You just need to run the following command in your project's folder.' Below this text is a code block with the following command:

```
mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
-Dsonar.projectKey=factorial-back-2026 \
-Dsonar.projectName='factorial-back' \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=sqp_c50b3db63acf7bf4da5903778ab7b3c9b6e5845e
```

 To the right of the code block is a 'Copy' button. At the bottom, there is a link to the 'official documentation of the Scanner for Maven'.

FIGURE 13 – Créer un test local.

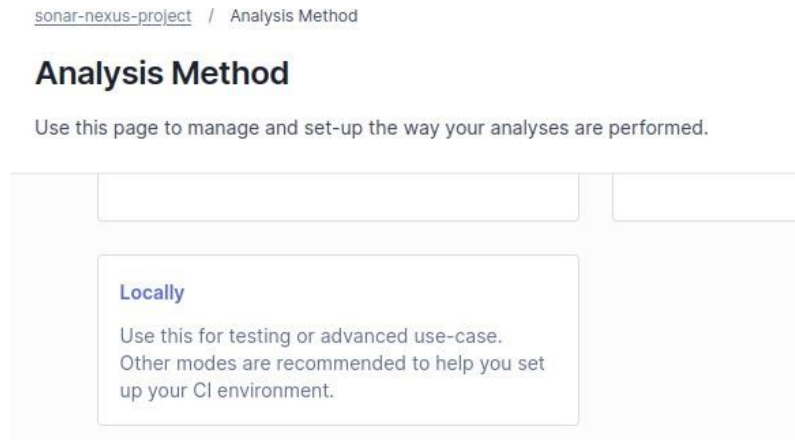


FIGURE 14 – Écran de préparation de l'analyse backend.

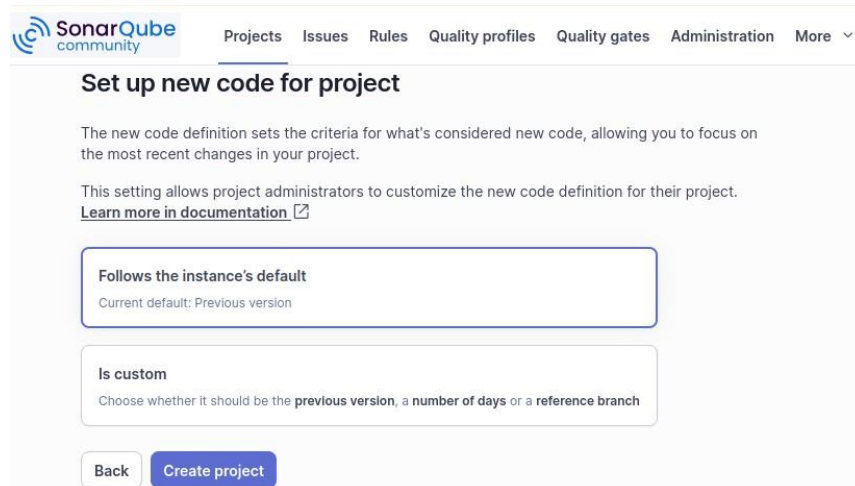


FIGURE 15 – Choix de la méthode d'analyse du backend.

6.2 Génération du token d'accès

SonarQube nécessite un token pour authentifier les analyses lancées depuis la machine locale ou depuis un pipeline CI/CD. Le token est généré depuis My Account, puis Security, dans la section Tokens. Une durée de validité peut être définie, par exemple 30 jours, 90 jours, ou aucune expiration selon la politique de sécurité choisie.

Generate a project token Use existing token

Token name * ⓘ Expires in

Analyze "sonar-nexus-project" 30 days Generate

ⓘ Please note that this token will only allow you to analyze the current project. If you want to use the same token to analyze multiple projects, you need to generate a global token in your [user account](#). See the [documentation](#) for more information.

FIGURE 16 – Génération d’un token d’accès dans SonarQube.

6.3 Choix de la méthode d’analyse

La méthode d’analyse sélectionnée est **Locally**. Dans l’étape suivante, l’option **Use existing token** est choisie afin d’utiliser le token généré précédemment. Maven est ensuite sélectionné comme outil de build pour lancer l’analyse du backend.

1 Provide a token

Analyze "sonar-nexus-project": **sqp_5bb5240aa0396b3dce1a6bc2040815400f256cb7**

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your [user account](#).

Continue

FIGURE 17 – Utilisation d’un token existant pour l’analyse locale.

Execute the Scanner for Maven

Running a SonarQube analysis with Maven is straightforward. You just need to run the following command in your project's folder.

```
mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
-Dsonar.projectKey=sonar-nexus-project-2026 \
-Dsonar.projectName='sonar-nexus-project' \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=sqp_5bb5240aa0396b3dce1a6bc2040815400f256cb7
```

Copy

FIGURE 18 – Commande générée par SonarQube pour le scan backend.

```
useradm@devops:~/Bureau/project-sonar-nexus$ mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
-Dsonar.projectKey=sonar-nexus-project-2026 \
-Dsonar.projectName='sonar-nexus-project' \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=sqp_5810e3411880fca372238e55ad62385e9baa9112 \
-Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
```

FIGURE 19 – Exécution de la commande d’analyse SonarQube pour le backend Spring Boot.



FIGURE 20 – Résultat du scan backend avant l’augmentation de la couverture de tests.

6.4 Amélioration après ajout des tests

Après l’ajout des tests, une nouvelle analyse est exécutée. Cette étape permet de mesurer l’impact des tests sur la qualité du projet, notamment sur la fiabilité et la couverture. L’intégration de tests dans la chaîne CI/CD améliore la confiance avant publication du livrable.



FIGURE 21 – Résultats après ajout et exécution des tests.



FIGURE 22 – Synthèse des métriques backend après analyse.

7 Mise en place de Nexus Repository

7.1 Rôle de Nexus

Nexus Repository est utilisé comme gestionnaire d’artefacts. Il centralise les livrables générés par les builds et facilite leur réutilisation dans les environnements de développement, de test, de préproduction ou de production.

Dans ce projet, Nexus est utilisé pour :

- stocker le livrable backend Spring Boot au format `.jar` ;
- stocker le livrable frontend Angular sous forme d’archive `.zip` ;

- organiser les dépôts selon leur rôle : hosted, proxy et group ;
- sécuriser l'accès aux artefacts par authentification.

```

useradm@devops:~/Bureau/project-sonar-nexus$ docker run -d \
--name nexus \
-p 8081:8081 \
-v nexus-data:/nexus-data \
sonatype/nexus3:latest
Unable to find image 'sonatype/nexus3:latest' locally
latest: Pulling from sonatype/nexus3
94eef8bbba5e: Pull complete
6a0ac1617861: Pull complete
045ff0e591b1: Pull complete
f2ef38c9664d: Pull complete
89557f3a9b6f: Pull complete
ba2054c6ed1e: Pull complete
2c1032d980bf: Pull complete
e2ff2228cef8: Pull complete
b33b3c81117e: Download complete
Digest: sha256:e08d0236154d13fb2a4fb050d9553762a92261b72d92dea99d99bb38ece5cac5
Status: Downloaded newer image for sonatype/nexus3:latest
d14d421d5e3b0ef07e832b12e5dcf94aa7b161a5a216246bb05dd2ed6c61704b
useradm@devops:~/Bureau/project-sonar-nexus$ docker exec nexus cat /nexus-data/admin.password
1b12f59b-07ce-4625-a21c-1eb8ec006a4auseradm@devops:~/Bureau/project-sonar-nexus$

```

FIGURE 23 – Interface Nexus Repository utilisée pour la gestion des artefacts.

7.2 Packaging du backend Spring Boot

Dans le projet Spring Boot, le build Maven permet de générer le fichier `.jar`. Ce fichier représente le livrable backend qui sera exécuté puis publié dans Nexus.

```

useradm@devops:~/Bureau/ci-cd/project-sonar-nexus$ mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:nexus-sonar-project >-----
[INFO] Building 0.0.1-SNAPSHOT
[INFO]
[INFO] -----[ jar ]-----
[INFO]

```

FIGURE 24 – Exécution du build Maven dans le projet Spring Boot.

```

LAB_NEXUS.md LAB_SONARQUBE.md mvnw mvnw.cmd pom.xml README.md src target
useradm@devops:~/Bureau/ci-cd/project-sonar-nexus$ cd target/
useradm@devops:~/Bureau/ci-cd/project-sonar-nexus/target$ ls
classes                               nexus-sonar-project-0.0.1-SNAPSHOT.jar
generated-sources                     nexus-sonar-project-0.0.1-SNAPSHOT.jar.original
generated-test-sources                site
jacoco.exec                          surefire-reports
maven-archiver                        test-classes
maven-status
useradm@devops:~/Bureau/ci-cd/project-sonar-nexus/target$

```

FIGURE 25 – Exécution du fichier JAR généré pour le backend.

7.3 Packaging du frontend Angular

Pour la partie Angular, les dépendances sont installées et le projet est construit afin de générer les fichiers statiques de distribution. Ces fichiers sont ensuite compressés dans une archive `.zip`, ce qui facilite leur publication dans Nexus.

```
useradm@devops:~/Bureau/ci-cd/angular-factorial-app$ ng build
Node packages may not be installed. Try installing with 'npm install'.
Error: Could not find the '@angular/build:application' builder's node package.
useradm@devops:~/Bureau/ci-cd/angular-factorial-app$ npm install

added 473 packages, and audited 474 packages in 26s

113 packages are looking for funding
  run `npm fund` for details
```

FIGURE 26 – Préparation du projet Angular avant génération du livrable.

```
useradm@devops:~/Bureau/ci-cd/angular-factorial-app$ ls
angular.json  package.json      README.md          tsconfig.json
dist          package-lock.json src                 tsconfig.spec.json
node_modules  public            tsconfig.app.json
useradm@devops:~/Bureau/ci-cd/angular-factorial-app$ cd dist
useradm@devops:~/Bureau/ci-cd/angular-factorial-app/dist$ ls
factorial-app
useradm@devops:~/Bureau/ci-cd/angular-factorial-app/dist$ cd factorial-app/
useradm@devops:~/Bureau/ci-cd/angular-factorial-app/dist/factorial-app$ ls
3rdpartylicenses.txt  browser  prerendered-routes.json
useradm@devops:~/Bureau/ci-cd/angular-factorial-app/dist/factorial-app$ cd browser/
useradm@devops:~/Bureau/ci-cd/angular-factorial-app/dist/factorial-app/browser$ ls
favicon.ico  index.html  main-QNPAUJ4U.js  styles-5INURTS0.css
useradm@devops:~/Bureau/ci-cd/angular-factorial-app/dist/factorial-app/browser$
```

FIGURE 27 – Génération des fichiers de distribution Angular.

```
useradm@devops:~/Bureau/ci-cd/angular-factorial-app/dist$ zip -r factorial-app-complet.zip factorial-app/
adding: factorial-app/ (stored 0%)
adding: factorial-app/3rdpartylicenses.txt (deflated 72%)
adding: factorial-app/browser/ (stored 0%)
adding: factorial-app/browser/favicon.ico (deflated 72%)
adding: factorial-app/browser/styles-5INURTS0.css (stored 0%)
adding: factorial-app/browser/main-QNPAUJ4U.js (deflated 69%)
adding: factorial-app/browser/index.html (deflated 36%)
adding: factorial-app/prerendered-routes.json (stored 0%)
useradm@devops:~/Bureau/ci-cd/angular-factorial-app/dist$
```

FIGURE 28 – Compression du livrable frontend.

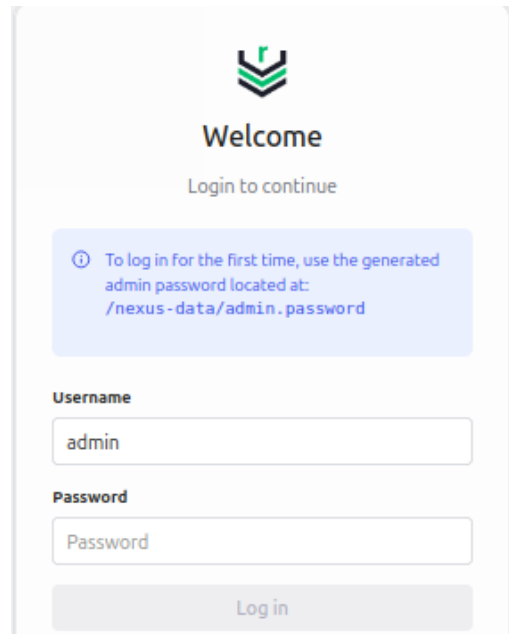


FIGURE 29 – Archive frontend prête à être publiée.

7.4 Authentification et accès à Nexus

L'accès à Nexus est protégé par un compte administrateur. Après connexion, il est possible de créer et d'administrer les dépôts nécessaires au projet.

```
adding: factorial-app/pre-rendered-routes.json (stored 0%)
useradm@devops: ~/Bureau/ci-cd/angular-factorial-app/dist$ docker exec nexus cat /nexus-data/admin.password
8178ffc4-850e-444f-83de-b10fed745008useradm@devops:~/Bureau/ci-cd/angular-factorial-app/dist$
```

FIGURE 30 – Commande ou information liée au mot de passe Nexus.

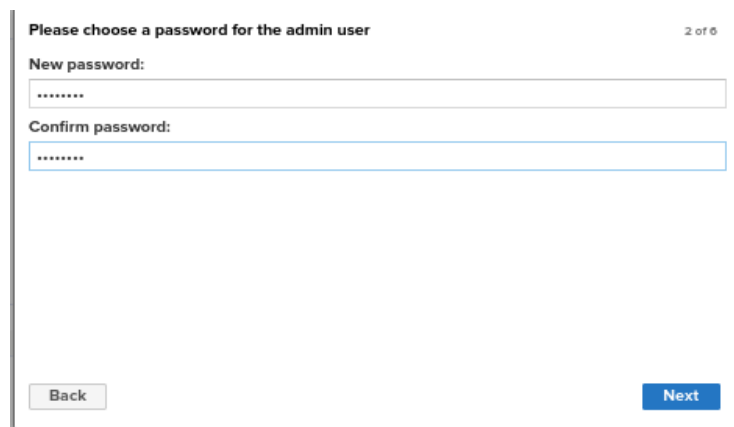
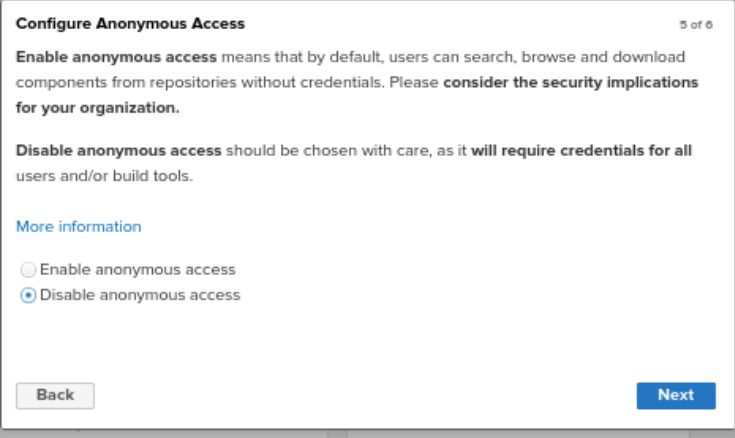


FIGURE 31 – Page de connexion à Nexus Repository.



Configure Anonymous Access 3 of 6

Enable anonymous access means that by default, users can search, browse and download components from repositories without credentials. Please **consider the security implications for your organization**.

Disable anonymous access should be chosen with care, as it **will require credentials for all** users and/or build tools.

[More information](#)

☐ Enable anonymous access

☒ Disable anonymous access

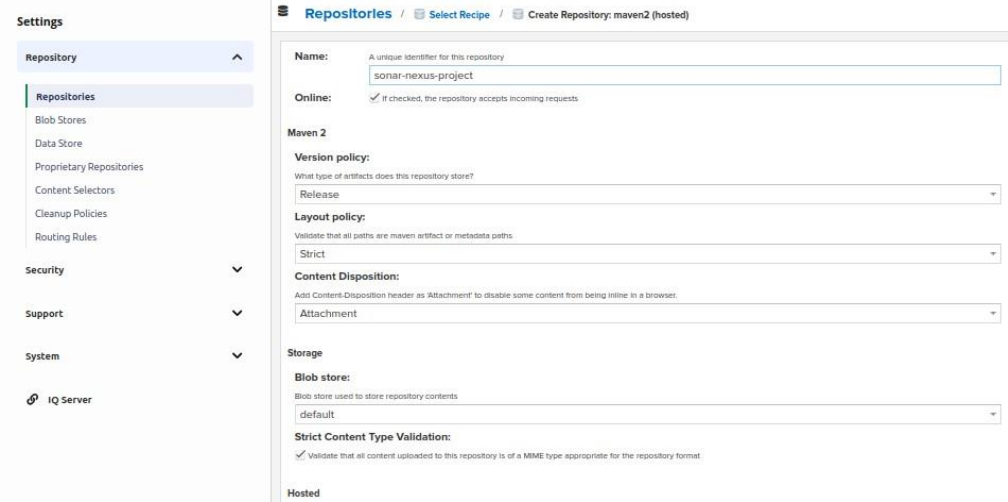
[Back](#) [Next](#)

FIGURE 32 – Interface d'administration après authentification.

8 Configuration des dépôts Nexus

8.1 Repository backend

Un dépôt dédié au backend est créé afin d'héberger les livrables Spring Boot. Le type de dépôt utilisé pour publier un artefact interne est généralement hosted. Ce dépôt devient la source officielle des versions backend générées par la chaîne CI/CD.



Settings

- Repository
- Repositories
- Blob Stores
- Data Store
- Proprietary Repositories
- Content Selectors
- Cleanup Policies
- Routing Rules
- Security
- Support
- System
- IQ Server

Repositories / Select Recipe / Create Repository: maven2 (hosted)

Name: A unique identifier for this repository
sonar-nexus-project

Online: ☒ If checked, the repository accepts incoming requests

Maven 2

Version policy:
What type of artifacts does this repository store?
Release

Layout policy:
Validate that all paths are maven artifact or metadata paths.
Strict

Content Disposition:
Add Content-Disposition header as 'Attachment' to disable some content from being inline in a browser.
Attachment

Storage

Blob store:
Blob store used to store repository contents
default

Strict Content Type Validation:
☒ Validate that all content uploaded to this repository is of a MIME type appropriate for the repository format

☒ Hosted

FIGURE 33 – Création ou configuration du repository backend.

Deployment policy:
Controls if deployments of and updates to artifacts are allowed
Disable redeploy

Proprietary Components:
☐ Components in this repository count as proprietary for namespace conflict attacks (requires Sonatype Nexus Firewall)

Cleanup

Cleanup Policies:
Components that match any of the Applied policies will be deleted

Available
Filter

Applied

Create repository Cancel

FIGURE 34 – Paramètres du dépôt backend.

Hosted

Deployment policy:
Controls if deployments of and updates to artifacts are allowed
Allow redeploy

Proprietary Components:
☐ Components in this repository count as proprietary for namespace conflict attacks (requires Sonatype Nexus Firewall)

Cleanup

FIGURE 35 – Dépôt backend disponible dans Nexus.

8.2 Repository proxy

Un dépôt proxy permet à Nexus de servir d'intermédiaire vers un dépôt distant. Il est utile pour mettre en cache les dépendances externes et améliorer la stabilité des builds, même lorsque le réseau ou le dépôt distant est temporairement indisponible.

Settings

- Repository
 - Repositories
 - Blob Stores
 - Data Store
 - Proprietary Repositories
 - Content Selectors
 - Cleanup Policies
 - Routing Rules
- Security
- Support
- System
- IQ Server

Repositories / Select Recipe / Create Repository: maven2 (proxy)

Maven 2

Version policy:
What type of artifacts does this repository store?
Release

Layout policy:
Validate that all paths are maven artifact or metadata paths
Strict

Content Disposition:
Add Content-Disposition header as 'Attachment' to disable some content from being inline in a browser.
Attachment

Proxy

Remote storage:
Location of the remote repository being proxied, e.g. https://repo1.maven.org/maven2/
https://repo.maven.apache.org/maven2/

Use the Nexus Repository truststore:
☐ Use certificates stored in the Nexus Repository truststore to connect to external systems View certificate

Blocked:
☐ Block outbound connections on the repository

FIGURE 36 – Configuration d'un repository de type proxy.

8.3 Repository group

Un dépôt group permet de regrouper plusieurs repositories sous une seule URL. Cette approche simplifie la configuration des clients, car Maven ou npm peut utiliser une URL unique pour accéder à plusieurs sources d'artefacts.

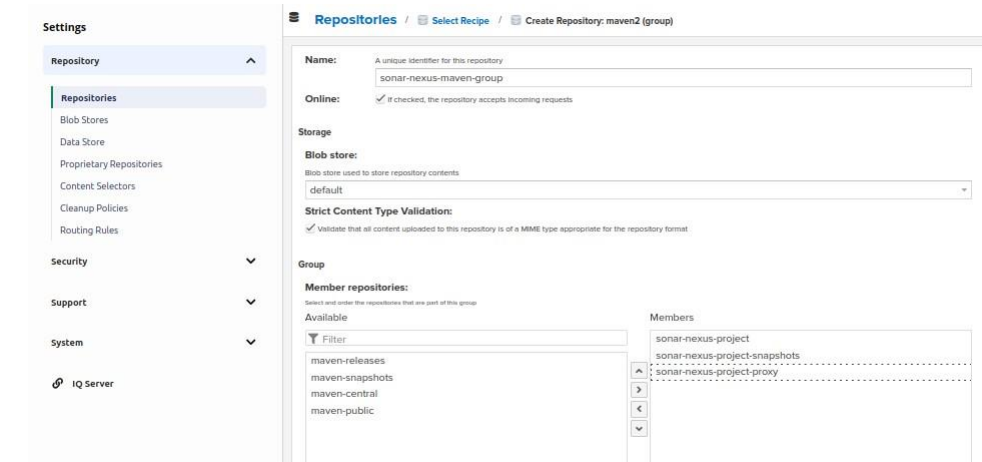


FIGURE 37 – Configuration d'un repository de type group.

sonar-nexus-maven-group	group	maven2	default	Online	copy			
sonar-nexus-project	hosted	maven2	default	Online	copy			
sonar-nexus-project-proxy	proxy	maven2	default	Online - Ready to Connect	copy	0	0	
sonar-nexus-project-snap...	hosted	maven2	default	Online	copy			

FIGURE 38 – Vérification du repository group créé.

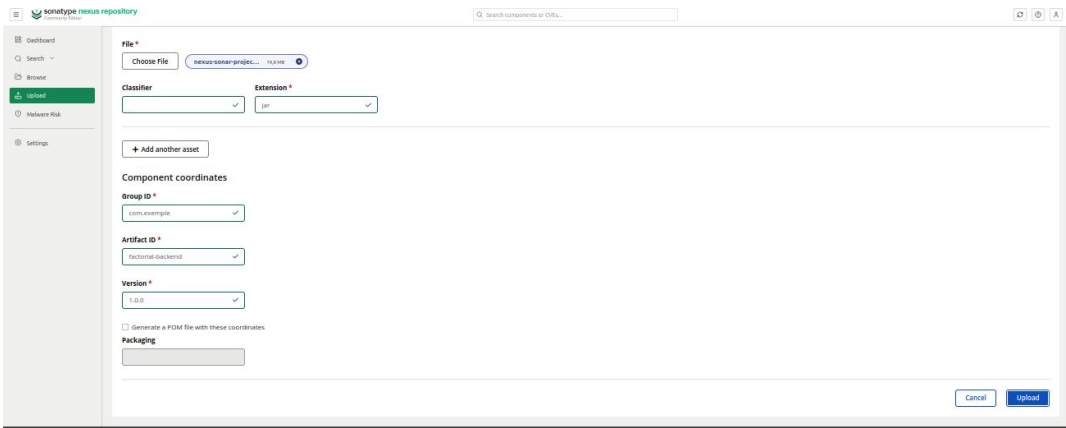


FIGURE 39 – uploader un fichier

Search

Keyword: com.example:factorial-backend

Enclose your criteria in quotation marks to search an exact phrase; otherwise, search criteria will be split by any commas, spaces, dashes, or forward slashes. All keyword searches automatically append a wildcard (*) at the end of each criterion.

Name	Group	Last Updated	Version	Format	Repository
factorial-backend	com.example	15/05/2020 21:51:31	1.0.0	maven2	myn.host

FIGURE 40 – vérification d'uploade

8.4 Repository frontend

Pour le frontend, un repository raw hosted est créé. Ce type de dépôt est adapté au stockage de fichiers génériques comme des archives .zip. L'archive générée à partir du build Angular est ensuite téléversée dans ce dépôt.

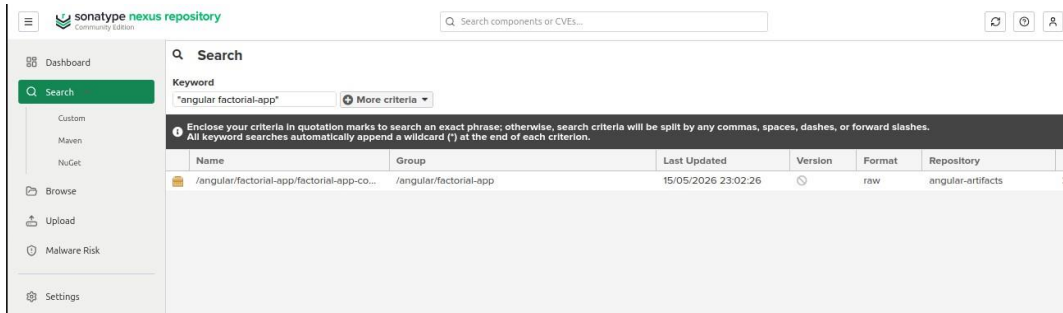


FIGURE 41 – Création du repository raw hosted pour le frontend.

9 Problème rencontré et solution appliquée

Lors du lancement de SonarQube, un problème peut apparaître à cause d'Elasticsearch. SonarQube utilise Elasticsearch en interne, et celui-ci peut échouer si le paramètre système `vm.max_map_count` est trop bas.

```
orker.run(ThreadPoolExecutor.java:614)\n java.base/java.lang.Thread.run(Thread.java:1474)
2026.05.14 11:46:50 INFO app[][o.s.a.SchedulerImpl] Process[Web Server] is stopped
2026.05.14 11:46:50 WARN app[][o.s.a.p.AbstractManagedProcess] Process exited with exit value [ElasticSearch]: 143
2026.05.14 11:46:50 INFO app[][o.s.a.SchedulerImpl] Process[ElasticSearch] is stopped
2026.05.14 11:46:50 INFO app[][o.s.a.SchedulerImpl] SonarQube is stopped
```

FIGURE 42 – Erreur Elasticsearch liée au paramètre système `vm.max_map_count`.

La solution consiste à augmenter cette valeur au niveau du système hôte avant de relancer le conteneur SonarQube. Ce réglage est important dans un environnement Linux, car Elasticsearch exige un nombre suffisant de zones mémoire mappées pour fonctionner correctement.

Deuxième partie : Intégration automatique avec Jenkins

ÉTAPE 1 — Préparer SonarQube pour Jenkins

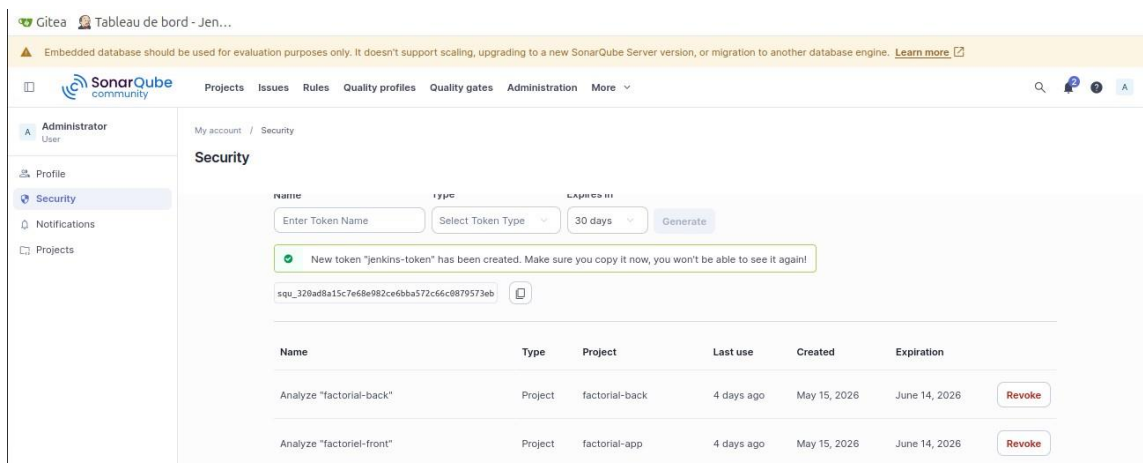
SonarQube – My Account – Security

→ Generate Token

Name : jenkins-token

Type : User Token

→ Copie : squ



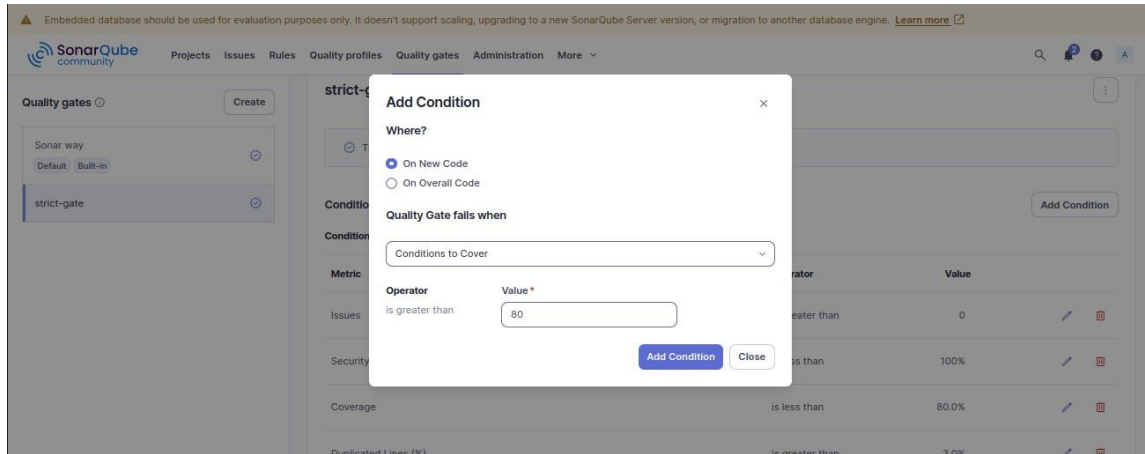
Créer un Quality Gate qui fait échouer le pipeline

SonarQube – Quality Gates – Create

Name : strict-gate

Add Condition :

Coverage is less than 80% : pipeline échoue si pas de tests



ÉTAPE 2 — Préparer Nexus pour Jenkins

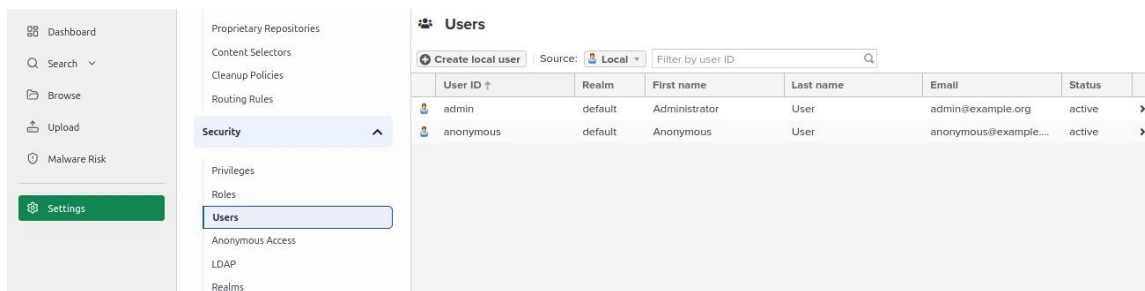
Créer l'utilisateur Jenkins dans Nexus

Nexus – Security – Users – Create ID

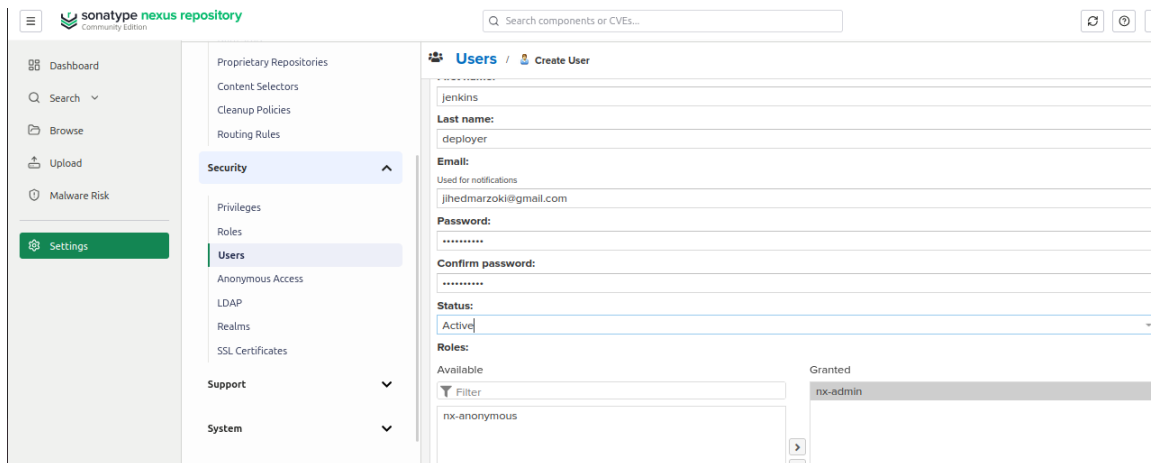
: jenkins-deployer

Password : jenkins123

Roles : nx-admin



Cree un user



ÉTAPE 3 — Configurer Jenkins

3.1 Installer les plugins nécessaires

Jenkins – Manage Jenkins – Plugins – Available

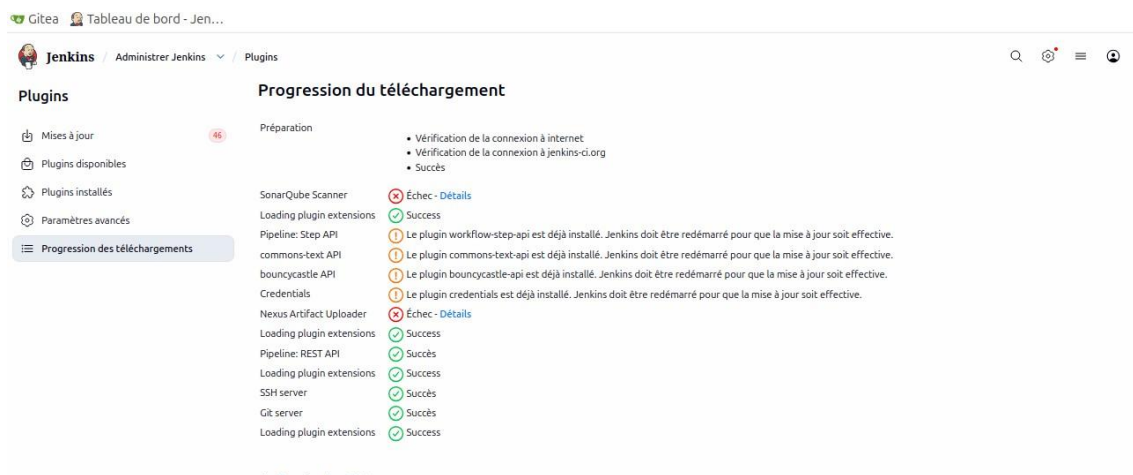
Installe :

SonarQube Scanner

Nexus Artifact Uploader

Pipeline

Git

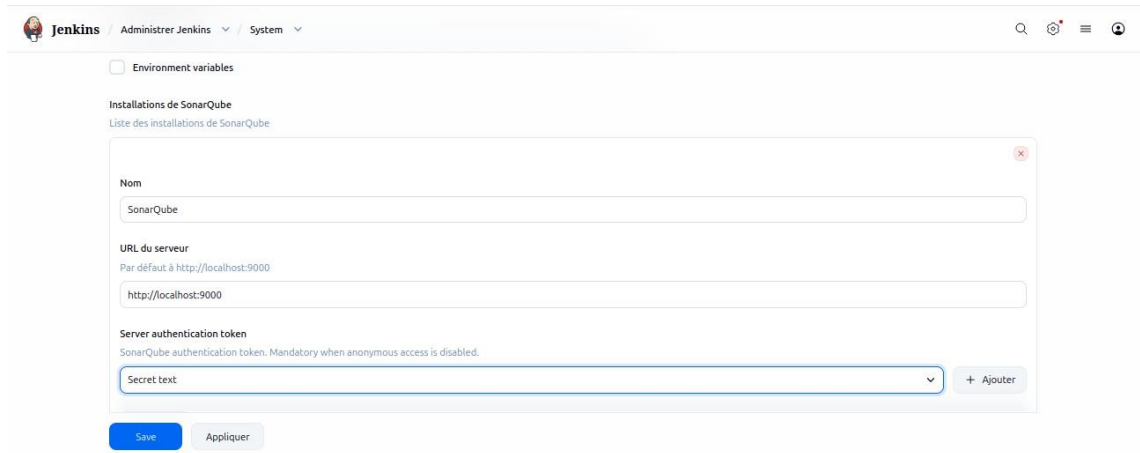


Jenkins – Manage Jenkins – System – SonarQube servers – Add

Name : SonarQube

URL : http://localhost:9000

Token : colle le token



The screenshot shows the Jenkins administration interface for configuring SonarQube installations. At the top, there's a navigation bar with 'Jenkins', 'Administrer Jenkins', and 'System'. Below this, a checkbox for 'Environment variables' is visible. The main section is titled 'Installations de SonarQube' with a subtitle 'Liste des installations de SonarQube'. A modal form is open for adding a new installation. It contains three fields: 'Nom' (Name) with the value 'SonarQube', 'URL du serveur' (Server URL) with the value 'http://localhost:9000' (and a default note 'Par défaut à http://localhost:9000'), and 'Server authentication token' (SonarQube authentication token, Mandatory when anonymous access is disabled) with a 'Secret text' dropdown. An '+ Ajouter' button is next to the token field. At the bottom of the modal, there are 'Save' and 'Appliquer' buttons.

Connecter Jenkins - Nexus

Jenkins –Manage Jenkins –Credentials - Add :

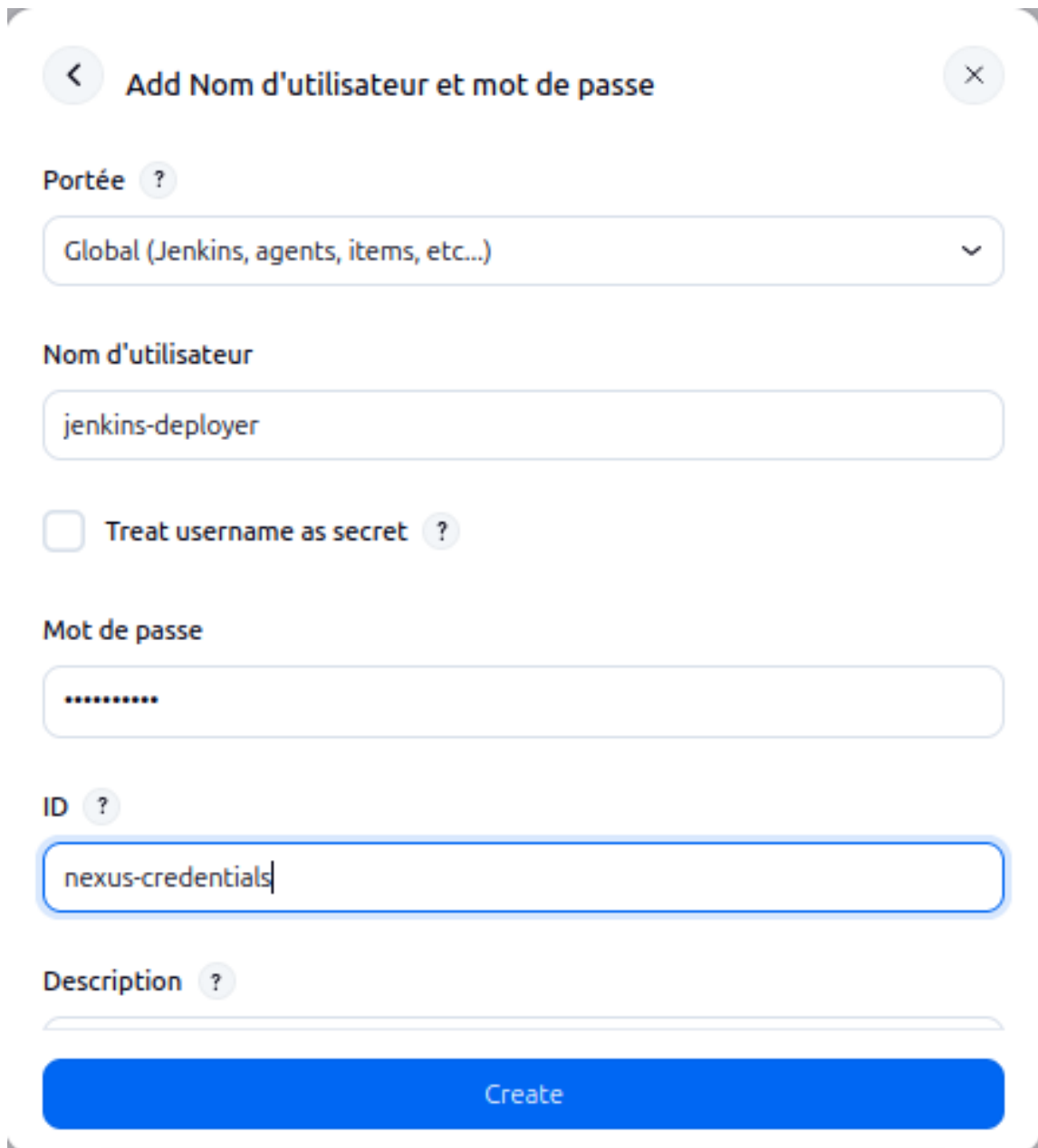
Username with password

Username : Jenkins-

deployer

Password : jenkins123

ID : nexus-credentials



The image shows a Jenkins configuration dialog box titled "Add Nom d'utilisateur et mot de passe". It contains several fields for creating a new user:

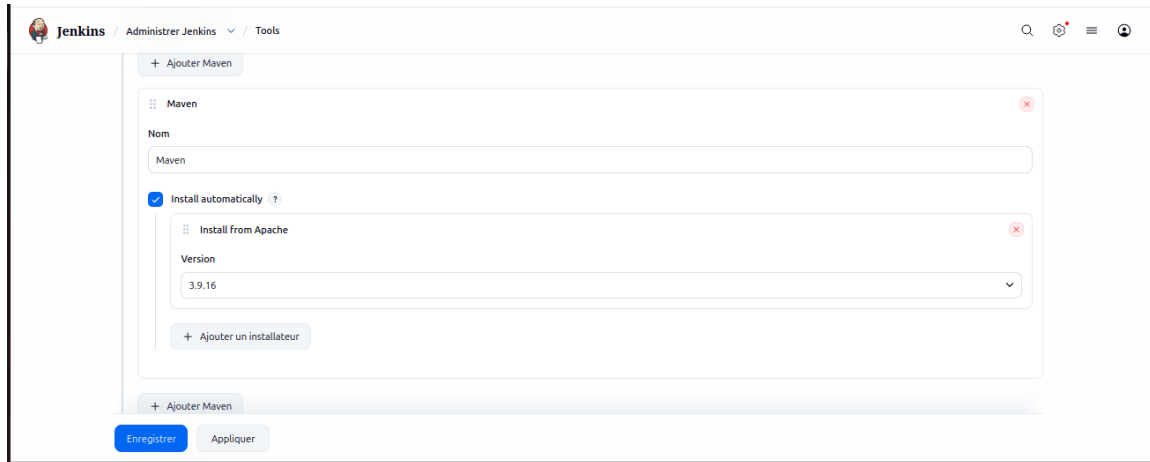
- Portée**: A dropdown menu with the selected value "Global (Jenkins, agents, items, etc...)" and a question mark icon.
- Nom d'utilisateur**: A text input field containing "jenkins-deployer".
- Treat username as secret**: An unchecked checkbox with a question mark icon.
- Mot de passe**: A password input field showing masked characters "*****".
- ID**: A text input field containing "nexus-credentials", which is highlighted with a blue border.
- Description**: An empty text input field with a question mark icon.
- Create**: A large blue button at the bottom right.

Configurer Maven dans Jenkins

Jenkins –Manage Jenkins – Tools –Maven Name :

Maven

Version : 3.9.6



Ajouter distributionManagement dans pom.xml

```
<distributionManagement>
  <repository>
    <id>nexus-releases</id>
    <url>http://localhost:8081/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>nexus-snapshots</id>
    <url>http://localhost:8081/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

Ajouter les credentials Nexus dans settings.xml

```
nano ~/.m2/settings.xml
```

```
<settings>
  <servers>
    <server>
      <id>nexus-releases</id>
      <username>jenkins-deployer</username>
      <password>jenkins123</password>
    </server>
    <server>
      <id>nexus-snapshots</id>
      <username>jenkins-deployer</username>
      <password>jenkins123</password>
    </server>
  </servers>
</settings>
```

Créer le Jenkinsfile

Dans le dossier de projet Spring Boot :

```
nano ~/Bureau/ci-cd/project-sonar-nexus/Jenkinsfile
```

```
pipeline {
  agent any
  tools {
    maven 'Maven'
  }

  options {
    timeout(time: 30, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '10'))
    timestamps()
  }

  stages {
    stage('Checkout') {
      steps {
        echo 'Récupération du code...'
        checkout scm
      }
    }

    stage('Build & Tests') {
      steps {
        echo 'Compilation et tests...'
        sh 'mvn clean package'
      }
    }
  }
}
```

Github user



Créer le job Jenkins

Jenkins – New Item

Name : factorial-pipeline

Type : Pipeline

–Pipeline section :

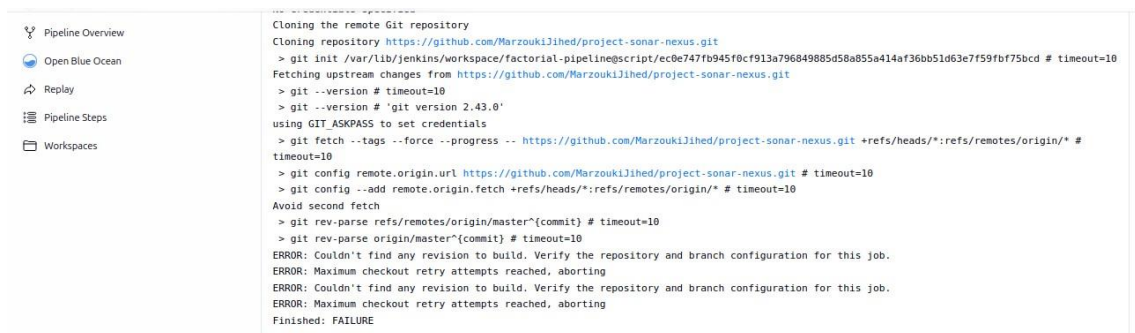
Definition : Pipeline script from SCM

SCM : Git

URL : <https://github.com/MarzoukiJihed/project-sonar-nexus.git>

Branch : main

Puis builder le pipeline :



Résultat du build Jenkins

 **factorial-pipeline**

SonarQube Quality Gate

factorial-back

Passed

server-side processing:

Success

Liens permanents

- [Dernier build \(#16\), il y a 13 mn](#)
- [Dernier build stable \(#16\), il y a 13 mn](#)
- [Dernier build avec succès \(#16\), il y a 13 mn](#)
- [Dernier build non réussi \(#15\), il y a 14 mn](#)
- [Dernier build complété \(#16\), il y a 13 mn](#)

11 Conclusion

Ce projet démontre la mise en place d'une chaîne CI/CD complète autour d'un projet Spring Boot et Angular. L'utilisation de SonarQube améliore la qualité du code grâce à une analyse continue, tandis que Nexus garantit une gestion centralisée et structurée des livrables.

La solution obtenue constitue une base solide pour une industrialisation plus avancée. Elle peut être enrichie par l'intégration d'un outil d'orchestration comme Jenkins, GitLab CI ou GitHub Actions afin d'automatiser entièrement les étapes de build, test, analyse SonarQube et publication dans Nexus.

Références utiles

- Documentation Sonatype sur les types de repositories : <https://help.sonatype.com/en/repository-types.html>
- Documentation Sonatype sur les raw repositories : <https://help.sonatype.com/en/raw-repositories.html>
- Documentation complémentaire sur Nexus, registre Docker privé et Docker Hub proxy : <https://mossaabfrifita.github.io/docs/devops/Cr%C3%A9ez%20un%20registre%20docker%20priv%C3%A9%20et%20un%20docker%20hub%20proxy%20avec%20Nexus>
- Discussion et support de rédaction utilisés pour l'organisation du rapport : <https://chat.deepseek.com/a/chat/s/04cfd72f-e03c-4c64-b721-573975f2e229>
- Interface SonarQube locale utilisée dans le projet : <http://localhost:9000>